

07. Test Environment Setup

7.1 Overview

The Test Environment Setup section describes the hardware, software, tools, frameworks, and configurations required to execute the OrangeHRM automation testing framework successfully.

The automation framework is developed using **Java, Selenium WebDriver, TestNG, and Maven** following the **Page Object Model (POM)** design pattern.

7.2 Hardware Requirements

Component	Specification
Processor	Intel Core i3 or above
RAM	Minimum 8 GB
Storage	Minimum 20 GB Free Space
Internet Connection	Stable Broadband Connection
Operating System	Windows 10 / Windows 11

7.3 Software Requirements

Software	Version
Java JDK	JDK 17 or above
Eclipse IDE	Latest Version
Selenium WebDriver	Latest Stable Version
TestNG	Latest Version
Maven	Latest Version
Google Chrome	Latest Version
ChromeDriver	Compatible with Chrome Browser
Git	Optional
GitHub	Optional

7.4 Application Under Test (AUT)

Item	Details
Application Name	OrangeHRM Demo
Application Type	Web Application
URL	https://opensource-demo.orangehrmlive.com
Testing Type	Functional Automation Testing

Item	Details
Framework Design Page Object Model (POM)	

7.5 Project Structure

src/main/java

This package contains all page classes and utility files.

Pages Package

1. HomePage.java

Purpose:

Handles Login Page functionalities.

Functions:

- Enter Username
 - Enter Password
 - Click Login Button
 - Capture Error Messages
-

2. DashboardPage.java

Purpose:

Handles Dashboard functionalities after successful login.

Functions:

- Verify Dashboard Visibility
 - Verify User Logged In
 - Dashboard Navigation
-

3. ForgotPasswordPage.java

Purpose:

Handles Forgot Password functionality.

Functions:

- Click Forgot Password Link
 - Enter Username
 - Submit Request
 - Cancel Request
-

4. SuccessForgotPasswordPage.java**Purpose:**

Handles Password Reset Success Screen.

Functions:

- Verify Reset Password Confirmation Message
 - Validate Success Screen Display
-

5. ApplyLeavePage.java**Purpose:**

Handles Leave Module functionalities.

Functions:

- Navigate to Leave Module
 - Select Leave Type
 - Select From Date
 - Select To Date
 - Enter Leave Reason
 - Submit Leave Request
-

Utilities Package

1. LoginMessages.java**Purpose:**

Stores Login Module expected messages.

Example Messages:

Invalid credentials
Required

Benefits:

- Centralized Message Management
 - Easy Maintenance
 - Reusability
-

2. ForgotPasswordMessages.java

Purpose:

Stores Forgot Password expected messages.

Example Messages:

Reset Password link sent successfully
Required

Benefits:

- Avoid Hard Coding
 - Easy Update
 - Better Maintainability
-

src/test/java

Contains all test execution classes.

Base Package

BaseTest.java

Purpose:

Common setup and teardown methods.

Responsibilities:

- Browser Launch
- Driver Initialization

- Open Application URL
- Close Browser
- Test Configuration

Annotations Used:

@BeforeMethod
@AfterMethod

ForgotPassword Package

ForgotPasswordTests.java

Purpose:

Executes Forgot Password related test cases.

Covered Scenarios:

- Valid Username
 - Invalid Username
 - Blank Username
 - Cancel Button Functionality
-

LoadPage Package

LoadPageTest.java

Purpose:

Validates page loading behavior.

Covered Scenarios:

- Login Page Load
 - Dashboard Page Load
 - Forgot Password Page Load
 - Apply Leave Page Load
-

Login Package

LoginTests.java

Purpose:

Executes Login Module test cases.

Covered Scenarios:

- Valid Login
 - Invalid Login
 - Empty Username
 - Empty Password
 - Empty Credentials
-

Navigation Package

NavigationTest.java**Purpose:**

Validates application navigation.

Covered Scenarios:

- Dashboard Navigation
 - Apply Leave Navigation
 - Menu Navigation
 - Submenu Navigation
-

7.6 Framework Design Pattern

Page Object Model (POM)

The framework follows the Page Object Model design pattern where each web page is represented as a separate Java class.

Advantages

- Better Code Reusability
 - Improved Maintainability
 - Reduced Code Duplication
 - Easy Script Enhancement
 - Better Readability
-

7.7 Browser Configuration

Browser Used

- Google Chrome

Browser Driver

- ChromeDriver

Browser Settings

- Browser Maximized
 - Cookies Enabled
 - Cache Cleared Before Execution (if required)
-

7.8 TestNG Configuration

Features Used

- Test Execution Control
- Assertions
- Reporting
- Test Prioritization

Sample Annotations

```
@BeforeMethod  
@Test  
@AfterMethod
```

7.9 Maven Dependencies

The project uses Maven for dependency management.

Major Dependencies

```
Selenium Java  
TestNG  
WebDriverManager
```

Benefits

- Automatic Dependency Download

- Version Control
 - Easy Build Management
-

7.10 Execution Process

Step 1

Open Eclipse IDE.

Step 2

Import Maven Project.

Step 3

Update Maven Dependencies.

Step 4

Execute TestNG Test Class.

Step 5

Observe Execution in Console.

Step 6

Verify Test Results.

Step 7

Review Pass/Fail Status.

7.11 Expected Outcome

After successful environment setup:

- Application launches successfully.
- Browser opens correctly.
- Selenium WebDriver initializes successfully.
- Login tests execute successfully.
- Forgot Password tests execute successfully.
- Navigation tests execute successfully.

- Apply Leave page validations execute successfully.
- Test execution results are displayed correctly.